

PLSQL

Name: Sravan Chandrakanth

Roll No: AM.SC.U4AIE23152

1. Do all the questions discussed in the class

```
create table emp1(eno int primary key,ename varchar(20),sal numeric(10,2),dno int)
create table dept1(dept_no int primary key,dname varchar(20))
```

```
insert into emp1 values
(1,'Paaripally V',10000,01),
(2,'Sravan S',15000,02),
(3,'kishore',20000,03),
(4,'Aashan',25000,02),
(5,'Rashid',18000,03);
```

```
insert into dept1 values
(01,'Public service'),
(02,'Kalari'),
(03,'Mukri');
```

```
create or replace function sum(a int,b int)
returns int as
$$
    declare
        c int;
    begin
        c := a+b;
        return c
    end;
$$
language 'plpgsql';
```

```
create or replace function odd_even(int)
return varchar as
$$
begin
if $1 % 2 == 0
return 'even';
else
return 'odd';
end if
$$
language 'plpgsql';
```

```
create or replace function emp_sal(empno emp1.eno % type)
returns emp1.sal %type as
$$
declare
salary emp1.sal %type;
begin
select sal into salary from emp1 where eno = empno;
return salary;
end;
$$
language 'plpgsql';
```

```

create or replace function sal_update(empno emp1.eno%type)
return void is
$$
declare
salary emp1.sal%type
begin
select sal into salary from emp1 where eno = empno;
if (sal < (select avg(sal) from emp1 where dno = (select eno from emp1 where eno = empno)));
end if,update emp set sal = sal + sal*1 where eno = empno;
end;
$$
language 'plpgsql';

create or replace function dept_name(empname emp1.ename%type)
return dept1.dname%type
$$
begin
return (select dname from dept1 where dno = (select dno from emp1 where ename = empname));
end;
$$
language 'plpgsql';

```

2. Consider the following schema .

Emp(eno, ename, sal)
EMP_PROJ(eno, pno, prjt_hrs).

- a) Write a function ProjectLoad that returns the total project working hours for the given eno.
- b)
- c) Write a PL/SQL code to update the salary of an employee if the employee earn less than the average salary.

New salary is current sal + difference between current sal and average salary

```

CREATE TABLE emp_proj (
    eno INT,
    pno INT,
    prjt_hrs INT,
    FOREIGN KEY (eno) REFERENCES emp1(eno)
);

--2a
CREATE OR REPLACE FUNCTION ProjectLoad(p_eno INT)
RETURNS INT
LANGUAGE plpgsql
AS $$
DECLARE
    v_total_hrs INT := 0;
BEGIN
    SELECT COALESCE(SUM(prjt_hrs), 0)
    INTO v_total_hrs
    FROM EMP_PROJ
    WHERE eno = p_eno;

    RETURN v_total_hrs;
END;
$$;

```

```

--2c
DO $$
DECLARE
    v_avg_sal NUMERIC;
BEGIN
    SELECT AVG(sal)
    INTO v_avg_sal
    FROM Emp;

    UPDATE Emp
    SET sal = v_avg_sal
    WHERE sal < v_avg_sal;
END;
$$;

```

3. Consider the following tables
Item(ino, iname, unit_price)
Transaction(tr_no,ino,qty)

Write a function to accept an item no from the user. If transaction has been made for less than 2 times for that item(check from transaction table), delete the item from the Item table.

```

--3
CREATE OR REPLACE FUNCTION check_and_delete_item(p_ino INT)
RETURNS TEXT
LANGUAGE plpgsql
AS $$
DECLARE
    v_count INT;
BEGIN
    SELECT COUNT(*)
    INTO v_count
    FROM transaction
    WHERE ino = p_ino;

    IF v_count < 2 THEN
        DELETE FROM item
        WHERE ino = p_ino;

        RETURN 'Item deleted because it had less than 2 transactions';
    ELSE
        RETURN 'Item NOT deleted. Transaction count = ' || v_count;
    END IF;
END;
$$;

```

4.

Consider a Bank database which includes the following tables.

ACCOUNTS(ac_no,br_no, cust_no,ac_type,bal)
BRANCHES(br_no, br_name, loc)
CUSTOMER(cno, cname, c_type)

- a) Write a function that accepts a threshold value and a customer number .
The program updates the c_type based on the threshold value. Ie. If balance > threshold then class A, else class B.
- b) Write a function called CloseBranch that takes two arguments (the branch to be closed and the branch to take over the accounts) and transfers all accounts at the closing branch to the new branch and removes the closing branch.
- c) Write a function that implements a "safe" withdrawal operation, that only permits a withdraw if there sufficient funds in the account to cover it.

```
--4a
CREATE OR REPLACE FUNCTION update_customer_type(p_threshold NUMERIC, p_cust_no INT)
RETURNS TEXT
LANGUAGE plpgsql
AS $$
DECLARE
    v_total_bal NUMERIC := 0;
BEGIN
    SELECT COALESCE(SUM(bal), 0)
    INTO v_total_bal
    FROM accounts
    WHERE cust_no = p_cust_no;

    IF v_total_bal > p_threshold THEN
        UPDATE customer
        SET c_type = 'A'
        WHERE cno = p_cust_no;
        RETURN 'Customer class updated to A';
    ELSE
        UPDATE customer
        SET c_type = 'B'
        WHERE cno = p_cust_no;
        RETURN 'Customer class updated to B';
    END IF;
END;
$$;
```

```
--4b
CREATE OR REPLACE FUNCTION CloseBranch(p_old_br INT, p_new_br INT)
RETURNS TEXT
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE accounts
    SET br_no = p_new_br
    WHERE br_no = p_old_br;

    DELETE FROM branches
    WHERE br_no = p_old_br;

    RETURN 'Branch closed and accounts transferred successfully';
END;
$$;
```

--4c

```
CREATE OR REPLACE FUNCTION safe_withdraw(p_ac_no INT, p_amount NUMERIC)
RETURNS TEXT
LANGUAGE plpgsql
AS $$
DECLARE
    v_balance NUMERIC;
BEGIN
    SELECT bal INTO v_balance
    FROM accounts
    WHERE ac_no = p_ac_no;

    IF v_balance IS NULL THEN
        RETURN 'Account does not exist';
    END IF;

    IF v_balance >= p_amount THEN
        UPDATE accounts
        SET bal = bal - p_amount
        WHERE ac_no = p_ac_no;

        RETURN 'Withdrawal successful. New balance = ' || (v_balance - p_amount);
    ELSE
        RETURN 'Insufficient funds. Withdrawal denied.';
    END IF;
END;
$$;
```